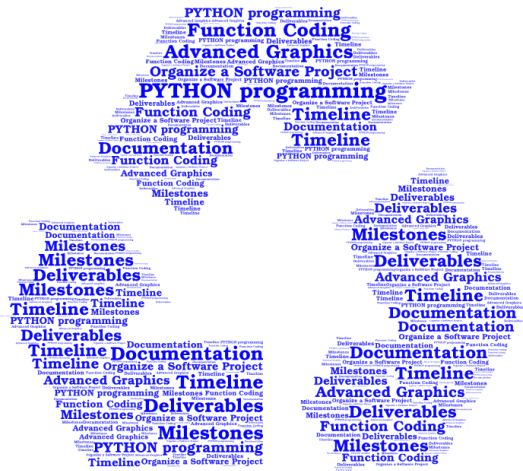




# Python Programming: Using Modules

Lecture: Modelling an Simulation

This is a compulsory elective module  
of  
**Technology of Circular  
Economy**  
(Master of Engineering)

**Dr. Uwe Reimer**  
Senior Scientist

## Recap from last time

Text encryption homework: did you achieve it?

Short summary, what did you learn so far:

- How to run a Python program (overview).
- Code writing (text indent, brackets, comments)

Using functions:

- What is a function?
- The namespace
- Given parameters, global statement, return values

Next week, I am out of office.

Therefore, there will be homework for you.

You can find the solution for task 1) and task 2) in the Moodle course.

Here, we will discuss the major features of the solution.

Your program for decoding should check all possible shifts and create some output.

You can read and understand the message if you found the 'correct' value for the shift.

The key for the solution is to create a loop around your decoding program.

You can decide to follow the last lecture and put the decoding part into a function.

This function then can be called from within a loop. But there are also other ways to do it.

The output of your program could look like (just a suggestion) this:

1

vjku ku cp gzcornq vgzv hqt vjg ngevwtg vgejpqnqia qh ektewnct geqpqoa!

-----

2

uijt jt bo fybnqmf ufyu gps uif mfduvsf ufdiopmphz pg djsdvmbz fdpopnz!

-----

3

this is an example text for the lecture technology of circular economy!

-----

# Solution 1

function declaration

loop over all letters (26)

```
12
13  + def mydec(mytext, shift):
33
34  shift = 1
35
36  - while shift < 26:
37      ... newtext = newtext + str(shift) + '\n' ... # here, we add the number
38      ... mydec(mytext, (-1) * shift)
39      ... shift = shift + 1
40      ... newtext = newtext + '\n' ... # we add t
41
```

function is called

1

kyv scrtb svri tre sv wfleu ze efikyvie rdviztr zk zj urexviflj! kyv scrtb svvi tre sv wfleu ze efikyvie vlifgv zk zj efk urexviflj!

-----

2

jxu rbqsa ruqh sqd ru vekdt yd dehjxuhd qcuhsyq yj yi tqdwuheki! jxu rbqsa ruuh sqd ru vekdt yd dehjxuhd ukhefu yj yi dej tqdwuheki!

-----

3

iwt qaprz qtpg rpc qt udjcs xc cdgiwtgc pbtgxrp xi xh spcvtdjh! iwt qaprz qttg rpc qt udjcs xc cdgiwtgc tjgdet xi xh cdi spcvtdjh!

-----

4

...

## Solution 2 – a)

add non-letter characters

```
16 def mydec(mytext, shift):
17     # This is a function to decode text.
18     global newtext
19     ....
20     along = 'abcdefghijklmnopqrstuvwxyz'
21     m = '.,?!' # non-letter characters in text
22     # First, we shift the positions
23     anew = along[shift:]
24     b = along[:shift]
25     anew = anew + b
26     along = anew
27     ....
28     # Here, we add the second alphabet in capital letters
29     # The simple way: along2 = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
30     along2 = along.swapcase() # this functions swaps the letters
```

add capital letters

## Solution 2 – b)

```
41 ..... for a in mytext:
42 .....     # inside this loop, 'a' contains the next letter from the text
43 .....     # We created a small and capital alphabet. For decryption we ne
        search in the respective alphabet
44 .....     if a.isupper():
45 .....         x = 0
46 .....         for b in alorg2: .....
47 .....             if b == a: ... # letter found at position x
48 .....                 newtext = newtext + alnew2[x] ... # new text uses same
49 .....                 x = x + 1
50 .....
51 .....     else: ..... # search in lower case alphabet (this also incl
52 .....         x = 0
53 .....         for b in alorg: .....
54 .....             if b == a: ... # letter found at position x
55 .....                 newtext = newtext + alnew[x] ... # new text uses same
56 .....                 x = x + 1
```

search in alphabet with capital letters

search in alphabet with lower case letters



This is the end of the text encryption example for programming.

What have you learned by now:

- How to write a Python program with a text editor.
- How to run a Python program (batch file / Windows PowerShell / online with Jupyter).
- How to work with strings.
- How to use files to read and write data.
- How to code loops.
- How to use the if statement.

From a previous lecture we know that functions are like small programs inside programs.

For larger programs you can distribute the code over several files. This make it more easy to handle and you can test the functionality step by step.

How does it work? Let's use the example from the previous lecture on functions (example 5).

```
1 b = 'hello'
2 x = 10 # number to repeat string
3
4 def myprint(b, x):
5     # this function prints a string x times
6
7     a = 0
8     while a < x:
9         a = a + 1
10        print(a, b)
```

Saves as myfunc.py

Create a new Python file in the same directory, e.g. mod-test.py

```
1 # This is an example to demonstrate
2 import myfunc
3 print(myfunc.b)
4 myfunc.myprint('lecture', 3)
```

← import program

← access to local variables

← access to functions

... and you do not need to use the actual file name. We can rename it.

Renaming 'myfunc' to 'a' in order to have a short name.

```
1 # This is an example to  
2 import myfunc as a  
3 print(a.b)  
4 a.myprint('lecture', 4)
```

← import program

← access to local variables

← access to functions

Modules are code files which can be loaded. This is a very general concept.

Python provides several modules as built-in (no extra installation). Examples are:

- **math module** (math constants, trigonometric functions, logarithm, power, ...)
- **cmath module** (the same as above for working with complex numbers)
- **random module** (generate a variety of pseudo random numbers)
- **statistics module** (statistics)
- **requests module** (make a request to a web page)



Links:

[https://www.w3schools.com/python/module\\_math.asp](https://www.w3schools.com/python/module_math.asp)

<https://docs.python.org/3/library/math.html>

This is very easy to use.

```
1 import math
2 print(math.pi)
```

There are also other packages for graphics and numerical tasks, but this is a story for the next lecture.